

Caso Netflix: questões para discussão

Aluno: Pedro Artur Jovanuzzi

Disciplina: Arquitetura de Software, Módulo 3: Serviços

Caso: Netflix: sete anos para reconstruir, dois segundos para começar o filme

1. Segundo o anúncio oficial, o que aconteceu em 2008 e qual operação da empresa ficou parada por três dias?

Em agosto de 2008 o banco de dados relacional da Netflix foi corrompido. Era um banco único, escalado verticalmente, no datacenter da própria empresa. O anúncio oficial descreve o episódio assim: "We experienced a major database corruption and for three days could not ship DVDs to our members."

A operação que parou foi o envio de DVDs pelo correio, o principal negócio da empresa naquele momento. Foram três dias sem despachar disco nenhum.

O que chama atenção não é a duração, e sim o fato de tudo ter parado junto. Não houve degradação parcial nem clientes atendidos enquanto outros esperavam: existia um componente só, e a perda dele levou a operação inteira. Dali saíram duas conclusões, uma sobre o desenho e outra sobre a infraestrutura: escalar verticalmente um banco único não ia dar certo, e comprar servidores no ritmo do streaming era um problema que a empresa não queria ter.

2. A Netflix declara ter reconstruído a tecnologia em vez de transportá-la. Explique a diferença entre as duas coisas e o que cada uma muda na arquitetura resultante.

Transportar é levar os sistemas do jeito que estão para a infraestrutura de um provedor. Muda o dono do datacenter e a forma de pagar, mas a arquitetura continua a mesma: o banco central continua central e o ponto único de falha de 2008 vai junto, agora hospedado na AWS. Reconstruir foi o que a Netflix diz ter feito, com "a cloud-native approach, rebuilding virtually all of our technology": o banco central virou armazenamentos NoSQL, um por serviço, e o monólito foi quebrado em serviços independentes.

A diferença aparece em pontos concretos. Com banco compartilhado a separação dos dados depende da disciplina de quem escreve o código; com um banco por serviço ela vira característica da estrutura. Com um componente central qualquer falha derruba o conjunto; com serviços separados o sistema degrada em partes. E a mudança deixa de ser feita no sistema inteiro para ser feita um serviço por vez.

O preço foi tempo: sete anos, de 2008 a janeiro de 2016. Em troca vieram os números que a empresa divulga, como oito vezes mais assinantes, disponibilidade perto de quatro nove e custo por início de reprodução bem menor, além do lançamento em mais de 130 países de uma vez, que só foi possível porque o serviço já rodava em várias regiões. A lição é que a nuvem entrega elasticidade e custo variável; disponibilidade e resiliência continuam vindo da

arquitetura.

3. O Open Connect inverte a lógica de cache: o conteúdo chega antes do pedido. Que propriedade do negócio da Netflix torna isso possível, e que tipo de serviço jamais conseguiria fazer o mesmo?

Um cache comum é reativo: o primeiro que pede paga o custo de buscar na origem. O Open Connect é o contrário, com preenchimento noturno dos aparelhos e pré-carga do conteúdo apropriado para cada região. Quando o assinante aperta o play, o vídeo já estava a poucos quilômetros dele.

Isso funciona porque o catálogo é finito, conhecido com antecedência e estável no curto prazo, já que o conteúdo é pré-gravado e licenciado antes de entrar no ar. Junte a isso uma demanda previsível, em que poucos títulos respondem por boa parte das horas assistidas dentro de cada região, e a taxa de acerto se mantém alta mesmo num aparelho de até 120 TB. Vale lembrar que a recomendação influencia cerca de 80% das horas assistidas, ou seja, a empresa não só prevê o que vai ser assistido como participa dessa decisão.

Serviços em que o conteúdo não existe antes do pedido nunca conseguiriam fazer o mesmo: transmissão ao vivo, videochamada, mensageria e qualquer resposta calculada por requisição, como busca, feed ou saldo bancário. Conteúdo gerado por usuário também não fecha a conta na cauda longa, porque um vídeo com poucas visualizações por mês não justifica ocupar espaço em milhares de aparelhos. O critério é sempre o mesmo: cache proativo só compensa quando o conjunto é pequeno, conhecido antes do pedido e a demanda é concentrada. É uma característica do negócio, não da tecnologia, e por isso não se copia junto com o software.

4. Compare Isthmus, arquitetura ativa-ativa e Chaos Kong quanto ao modo de falha que cada um endereça.

O Isthmus, de 2013, resolve um modo de falha específico: a queda do balanceador de carga de uma região, com o tráfego de entrada sendo roteado por outra região enquanto o processamento fica onde estava. É uma resposta sob medida e não cobre a perda da região inteira.

A arquitetura ativa-ativa, de dezembro de 2013, tem outra natureza. Não é remendo para um modo de falha, é uma capacidade: o serviço roda em mais de uma região ao mesmo tempo e cada uma consegue atender os membros da outra. Para isso a equipe teve que resolver replicação do Cassandra entre regiões, invalidação de cache no EVCache e roteamento dinâmico no Zuul. O ganho é cobrir qualquer falha regional, inclusive as que ninguém previu.

O Chaos Kong não endereça modo de falha nenhum, ele é o ensaio: evacua uma região com tráfego real e observa se a outra absorve. Só foi possível depois da ativa-ativa, e foi ele que revelou o problema seguinte, que no papel não aparecia, de a evacuação levar cerca de 50 minutos. O Project Nimble baixou isso para 8 minutos com duas pessoas em seis meses. Resumindo: um cobre um componente conhecido, o outro cobre a perda da região, e o terceiro mede se os dois realmente funcionam.

5. O Chaos Monkey exige a plataforma Spinnaker e o Hystrix está em modo de manutenção desde 2018. Explique o que cada um desses fatos diz sobre reaproveitar a plataforma de ferramentas de outra empresa.

O repositório do Chaos Monkey avisa: "You must be managing your apps with Spinnaker to use Chaos Monkey to terminate instances." A ferramenta assume a plataforma de entrega contínua da Netflix e, com ela, instâncias descartáveis, auto scaling repondo o que morreu, serviços sem estado local e observabilidade para perceber o efeito. Sem essa base, encerrar instâncias em produção não é teste de resiliência, é uma queda provocada pela própria empresa.

O Hystrix mostra o outro lado. Virou padrão de fato no ecossistema Java e está em modo de manutenção desde novembro de 2018, com o próprio repositório indicando o Resilience4j. Quem abriu o código seguiu adiante quando as necessidades mudaram, e não tem obrigação com quem adotou a biblioteca no meio do caminho.

Os dois fatos dizem a mesma coisa: a plataforma de ferramentas de uma empresa é subproduto do contexto dela, não um produto pronto para levar. O que dá para reaproveitar é o raciocínio, ou seja, por que precisaram de um disjuntor e que condições tornam o teste de caos seguro. O padrão sobrevive e a implementação é substituível. Vale desconfiar também do prazo de validade: a plataforma de 2012 ainda é apresentada como se fosse o estado atual da engenharia da Netflix, anos depois de partes dela terem sido aposentadas.